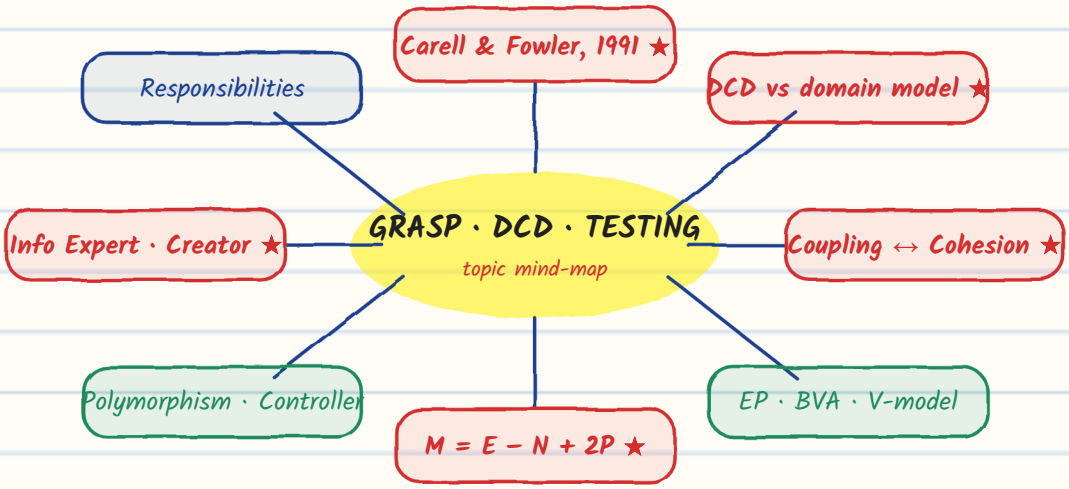
Date: 11.10.2026
topper's notes

GRASP, DCD & Testing Techniques

Responsibilities · nine guidelines · design class diagram · white/black box · cyclomatic complexity: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

➜ Syllabus coverage

"... Designing Objects with responsibilities. GRASP patterns with General Principles in assigning responsibilities: Information expert, Creator, Low Coupling and High Cohesion, Creating design class diagrams and mapping design to codes. Software Testing: White Box and Black Box testing. Basis Path Testing."

High-yield list

① GRASP = Carell & Fowler, 1991
② Info Expert = the class with the data
③ Creator = the container
④ Least dependent class wins
⑤ DCD adds operations + navigability
⑥ M = conditions + 1

Legend yellow = key term · red circle = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

➜ Contents

✓ GRASP: origin & the guidelines ..... p. 2
✓ Information Expert & Creator ..... p. 3
✓ Low Coupling & High Cohesion ..... p. 4
✓ The remaining guidelines ..... p. 5
✓ DCD: domain model vs design ..... p. 6
✓ DCD worked example + code ..... p. 7
✓ White-box vs black-box, EP, BVA ..... p. 8
✓ Coverage, cyclomatic, basis path ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

1 GRASP: the Responsibility-Assignment Problem

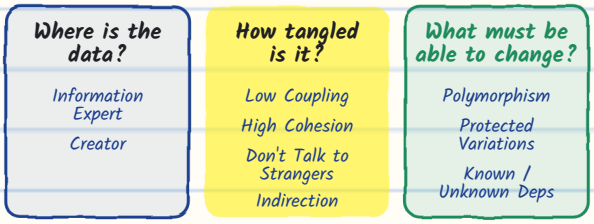
MCQ + Written

Definition **GRASP** = **G**eneral **R**esponsibility **A**ssignment **S**oftware **P**atterns: guidelines for deciding which object gets which responsibility, so that the result is loosely coupled, cohesive, maintainable and reusable. Proposed by James Carell and Martin Fowler, 1991 (*Toward Unified Design Methodologies*, IEEE Computer).

➜ Two kinds of responsibility

- **Knowing** — the object's own attributes and the objects linked to it.
- **Doing** — the actions it performs (its operations).

A **pattern** (Singleton, Strategy, Observer ...) is a named solution with structure. A GRASP guideline is one sentence with no class diagram attached — it tells you where to put a responsibility, not what to build.



the set sorted by the question each guideline answers

nine guidelines, three questions

➜ The nine guidelines, in one line each

#	Guideline	The principle, in one line
1	**Information Expert**	Give it to the class that **has the information**
2	**Creator**	The class that contains, aggregates or initialises it creates it
3	**Low Coupling**	Assign to the class **least dependent** on others
4	**High Cohesion**	One class, **maximum related** responsibility
5	**Polymorphism**	Do not re-implement superclass behaviour in a subclass
6	**Pure Arbiters**	The object that **arbitrates between** two or more others
7	**Don't Talk to Strangers**	Ask **only immediate associates** for information
8	**Controller**	The first **non-UI** object to receive a system event
9	**Intermediary** (Pure Fabrication)	An **invented**, non-domain class breaks the dependency

Three more often listed with GRASP

Indirection — an intermediary links objects that must not know each other.
Protected Variations — wrap every point of **predicted change** in an interface.
Known / Unknown Dependencies — name a **known** dependency concretely; an interface in front of a **changeable** one.

Exam trap GRASP is **guidelines for assigning responsibilities** by **Carell & Fowler (1991)** — not a pattern catalogue, and not the GoF patterns. **Pure Arbiters** is sometimes printed as "Pure Arbiters / Information Expert": do not count it twice.

2 Information Expert & Creator — Worked

high-yield

Information Expert Assign the responsibility to the object that has the information needed to carry it out. **Example:** the Invoice should compute getTotal(), because the Invoice is the only class holding its list of LineItems.

```
class LineItem {
    Product product;
    int quantity;
    double getSubTotal() {
        return product.price * quantity;
    }
}
class Invoice {
    List<LineItem> items;
    double getTotal() {
        return items.stream()
            .mapToDouble(LineItem::getSubTotal)
            .sum();
    }
}
```

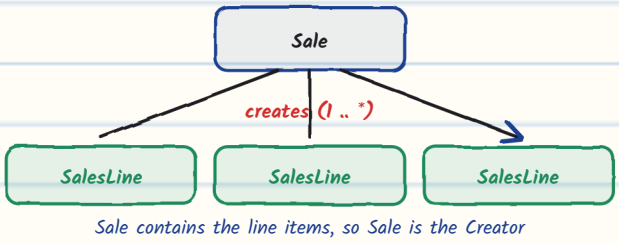
→ Why this is the right answer

- Only the **Invoice** holds the line items, so only it can sum them.
- No parameters have to be passed in and no LineItem needs a back-pointer.
- The knowledge stays inside the class that owns the data: coupling falls, cohesion rises.

Exam trap Information Expert is **not** "the class with the most methods". It is the class with the data needed. If the data lives elsewhere, the responsibility goes elsewhere.

→ 2b Creator — Worked

Creator Assign to class **B** the job of creating instances of class **A** if any one of these holds: B **contains** A · B **aggregates** A · B **records** A · B **closely uses** A · B **holds the data** that would initialise A.



Sale contains the line items, so Sale is the Creator

Creator: the container creates what it contains

```
class Sale {
    List<SalesLineItem> lines;
    SalesLineItem addLine(Product p, int q) {
        SalesLineItem l = new SalesLineItem(p, q);
        lines.add(l);
        return l;
    }
}
```

Memory trick **Creator = "C for Container"**. Even when B does not contain A, if B can name the values that initialise every field of A, B is a natural Creator — creating A there **reduces** the arguments the caller must supply.

Exam trap Creator is not "the class that uses it". In a POS, **Sale** — not **Register**, **ProductCatalog** or **Payment** — creates **SalesLineItem**, because Sale contains the line items.

3 Low Coupling & High Cohesion — Worked

written favourite

Low Coupling Assign responsibilities so that unnecessary dependencies are minimised. The rule to quote:
~~give the responsibility to the class least dependent on other classes~~.
Dependence is measured by how much of another object's **state** a class must know.

→ Bad — high coupling

```
class Customer {
    String name;
    void saveToDatabase() {
        // opens a JDBC connection,
        // builds SQL, commits a tx
    }
}
```

Cost: *Customer* now knows the DB, SQL, the driver and the network.

→ Good — low coupling

```
class CustomerRepository {
    void save(Customer c) {
        // all DB logic lives here
    }
}
class Customer {
    String name;  // no DB knowledge
}
```

Gain: *Customer* is reusable, the repository replaceable, the two unit-test separately.

→ 3b High Cohesion — Worked

```
// LOW COHESION — the "god class"
class CustomerManager {
    void browseShop()  { }
    void chargeCard()  { }
    void shipOrder()   { }
    void sendEmail()   { }
    void updateDatabase(){ }
}
```

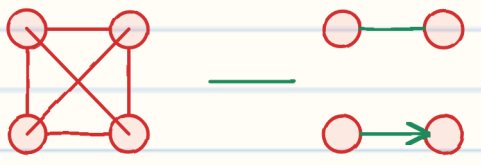
Five unrelated jobs: no single reason to change it, and every change risks all five.

```
// HIGH COHESION — split it
class Customer { /* customer data */ }
class Cart     { /* cart operations */ }
class Payment  { /* payment logic  */ }
class Shipping { /* delivery       */ }
class Notifier { /* e-mail         */ }
```

Each has one clear reason to change — SRP restated.

Memory trick **Coupling is how many strings are tied to this balloon** — four strings and it cannot fly. **Cohesion is how tightly its own parts stick together**: five unrelated jobs is gravel, not rock.

Exam trap The two are **different axes**: coupling is *between* classes, cohesion *within* one. A class can be cohesive yet coupled to three others.



HIGH COUPLING
every node knows every other node

LOW COUPLING
one hub, several independent spokes

the same four objects, two ways of connecting them

IT link A bank core hides its ledger rules behind an interface: the SQL is swappable without touching the business classes.

4 Polymorphism, Controller, Intermediary & the Rest

high-yield

Polymorphism Do **not** assign to a subclass a behaviour its superclass already covers. Put it in the superclass or interface and let polymorphism choose the version at run time. CashPayment · CardPayment · MobileBanking share process().

Pure Arbiters Assign a responsibility to the object that **arbitrates between two or more others** — it holds the rules both sides need and neither owns. A TaxPolicy that Invoice and Refund both consult.

Don't Talk to Strangers If a method knows too much about, or depends on, too many others, it breaks the rule. Ask **only immediate associates**. Never reach through Order to touch Order.getCustomer().getAddress().

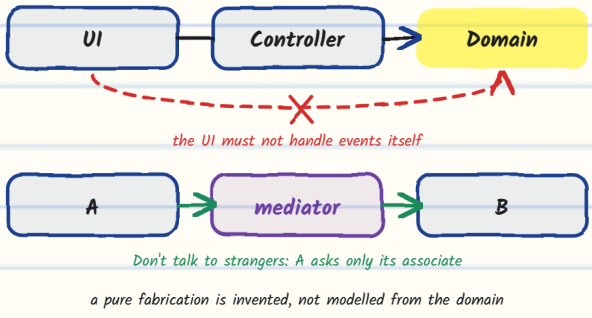
Controller The first object **beyond the UI layer** to receive a system event such as enterItem and coordinate it. Two kinds: **facade controller** (the whole system) and **use-case / session controller** (one scenario). Never in the UI class.

Intermediary (Pure Fabrication) A class that **represents no domain concept at all**, invented only to break an awkward dependency; normally placed in the **common low-level packages**. A ReportService between the invoice and the database.

Protected Variations Identify the points of **predicted change** and put an interface or abstract class in front of each, so the unstable part can vary without touching the rest. Open/Closed Principle; usually the Strategy pattern.

→ How the guidelines stack up against SOLID

GRASP guideline	It is really this principle
High Cohesion	Single Responsibility Principle
Low Coupling	Dependency Inversion · Open/Closed
Protected Variations	Open/Closed Principle · Strategy
Polymorphism	Liskov Substitution
Indirection / Intermediary	Dependency Inversion Principle



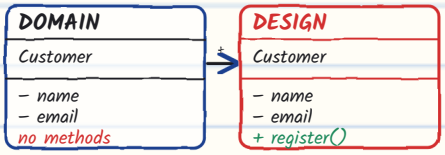
Controller sits beyond the UI; a mediator kills the shortcut

Exam trap The **Controller** guideline is **not** "the class the user clicks" — a UI class that receives the event is precisely what GRASP forbids. A **Pure Fabrication** differs from a **Domain Model** class: it has no domain meaning at all.

S The Design Class Diagram (DCD) MCQ + Written

Definition The **Design Class Diagram** is the UML class diagram drawn at the design level: the domain-model class diagram extended with everything an implementer needs — **operations** with signatures, **types**, **visibility** markers and **navigability** arrows.

Domain / analysis model	Design class diagram
Conceptual view of the domain	**Software** view
Classes, attributes, associations, multiplicities	those **plus methods** and return types
Plural domain nouns (Sales)	**Singular** nouns (Sale)
No software-specific types	Concrete: **String, double, List**
No visibility markers	**+ public · − private · # protected**
Associations are undirected	**Navigability** — who holds the reference
No interface or packages	**<<interface>>, <<abstract>>,** packages





the domain model keeps none of these

the same subject, drawn twice: conceptual, then designed

→ Mapping design to code: the one-to-one rules

Element in the DCD	Becomes in the code
One class box	One **class** — or an interface, if stereotyped <<interface>>
An attribute − price : double	One **field** / property of that type
An operation + total() : double	One **method** with that return type
An association 1 → * with navigability	A reference field plus a **collection** (array / List / Set)
Aggregation ◇ or composition ◆	A whole-part reference owned by the whole
A generalisation △ / realisation ▷	**inheritance** / composition by delegation

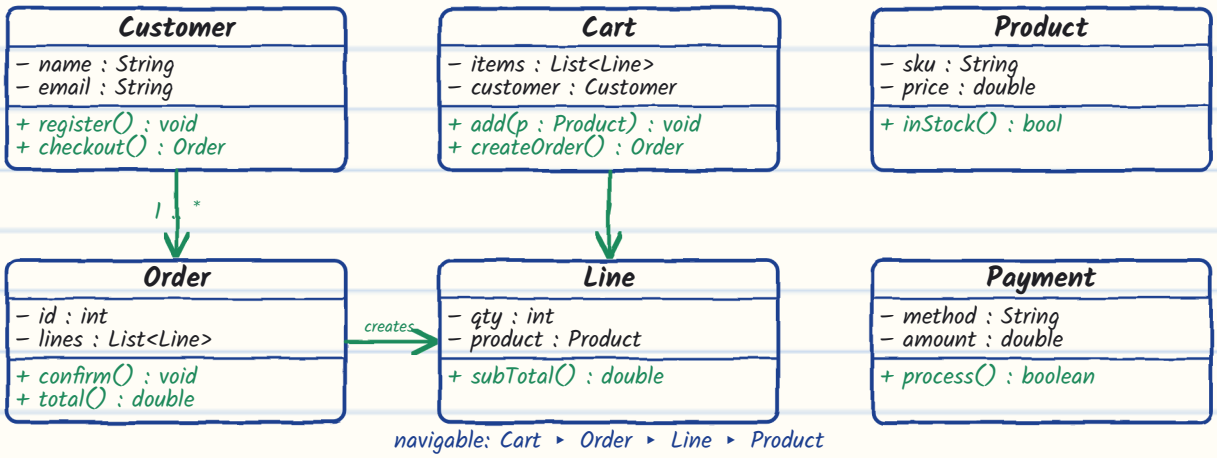
Exam trap The one thing a DCD shows and a domain model does **not** is ~~methods (operations) and navigability~~. Conceptual classes, associations and multiplicities appear in **both**.

Memory trick Domain model = **what exists**. DCD = **what exists, plus what it does**. A method signature with a return type means you are looking at a DCD.

Four steps to build a DCD ① Make every concept-class name **singular** (Sales → Sale). ② Give attributes **software types** (age: Integer → int). ③ Write each class's responsibilities as operations **with return types**. ④ Mark every association that holds a reference with a **navigability arrow**.

6 A DCD Worked Through, and the Code It Becomes

written favourite



*one design class diagram — Customer 1 — * Cart — * Order — * Line — * Product, plus a Payment collaborator; every box now carries operations with return types*

```
class Customer {
    String name;
    String email;
    Cart cart = new Cart();

    void checkout() {
        Order o = cart.createOrder();
        o.confirm();
    }
}
class Line {
    Product product;   // navigable ▸
    int qty;
    double subTotal() {
        return product.price * qty;
    }
}
```

```
class Cart {
    List<Line> items = new ArrayList<>();

    void add(Product p, int q) {
        items.add(new Line(p, q));
    }
    double calculateTotal() {
        return items.stream()
            .mapToDouble(Line::subTotal)
            .sum();
    }
    Order createOrder() {
        return new Order(items);
    }
}
```

Read the mapping off the picture

Cart.items : List<Line> → the field List<Line> items; Line.product : Product → the field Product product; Line.subTotal() : double → the method. ***Nothing in the DCD has to be re-invented in the code*** *— the class names, attribute names, types and method names already are the identifiers the programmer needs.*

Exam trap *A DCD is* ***not*** *the same artefact as the CRC card: CRC is how you discover responsibilities, the DCD is how you record them. "Design maps directly to code" means a* ***one-to-one mapping*** *— not one file per diagram element.*

7 Structural vs Functional Testing, EP and BVA

MCQ + Written

Aspect	White-box (structural, glass-box)	Black-box (functional, behavioural)
Test cases derived from	the **internal structure** of the code	the **specification**, not the code
Tester's knowledge	knows the program internals	does **not** know how it is written
Also called	structural, clear-box, glass-box	functional, specification-based
Main question	"is every statement / branch **executed**?"	"is the **output** right for every valid input?"
Techniques	basis-path testing, coverage analysis	equivalence partitioning, BVA, cause-effect graphing
Best at finding	dead code, unexecuted branches, wrong loop limits	missing requirements, wrong outputs

Test basis The document a test is derived from: for a **white-box** test the **design model / code**, for a **black-box** test the **requirement specification**. Testing can show the **presence** of defects, never their absence.

Exam trap Testing **cannot prove a program correct**; it only raises confidence. A test that has **never failed** is not yet known to be a good test. Unit → integration → validation → system levels, debugging, maintenance and test metrics belong to **Testing Strategies, Maintenance, Quality & Metrics**.

→ 7b Equivalence Partitioning & Boundary-Value Analysis

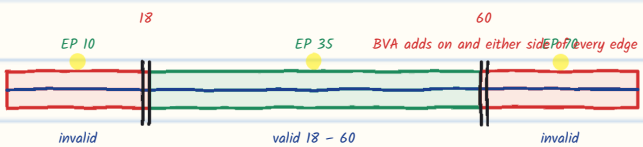
Equivalence class (age)	Kind	EP pick
age < 18	invalid	10
18 ≤ age ≤ 60	**valid**	35
age > 60	invalid	70

EP splits the input domain into classes handled the same way and tests **one representative** of each — few cases, wide coverage. Valid and invalid classes are counted **separately**.

Boundary	BVA picks
lower, 18	17 · **18** · 19
upper, 60	59 · **60** · 61

BVA tests **just below, exactly at and just above** each boundary. Off-by-one errors live at the edge — exactly where EP takes one arbitrary sample.

When to use which EP alone is enough for a small input domain. With a wide domain (years, amounts, dates) EP on its own grows risky, so add **BVA**: EP keeps the representative, BVA keeps the extreme, and a fault inside a class slips past EP, which always aims at the middle.



three equivalence classes; BVA adds the six values on the two edges

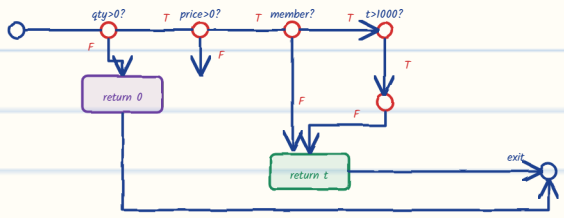
8 Coverage, Cyclomatic Complexity & Basis-Path Testing

written favourite

Coverage criterion	How it is measured	Strength
Statement	executed ÷ total executable statements × 100 %	weakest — misses unexecuted branches
Branch / decision	outcomes taken ÷ total outcomes × 100 %	the usual minimum target
Condition	atomic conditions evaluated ÷ total × 100 %	both ways for each part of a && b
Multiple condition	every truth-value combination — 2^n	very strong, very costly
Path	executed paths ÷ total paths × 100 %	strongest, exponential effort

Cyclomatic complexity (McCabe, 1976) $M = E - N + 2P$ — **E** = edges of the control-flow graph, **N** = nodes, **P** = connected components. A normal module gives **P = 1**, so the familiar form is **M = E − N + 2**. **M** is exactly the number of independent paths — the regions of the flow graph — and for straight-line conditionals **M = (independent conditions) + 1**. No decision at all ⇒ **M = 1**.

```
double total(int qty, double price,
             boolean member) {
  if (qty > 0 && price > 0) {
    double t = qty * price;
    if (member && t > 1000)
      t = t * 0.90;
    return t;
  }
  return 0.0;
}
```



E = 12 edges · N = 9 nodes M = 12 − 9 + 2 = 5 five independent paths

the control-flow graph: E = 12, N = 9, so five independent paths

1. Draw the flow graph
2. Compute **M**
3. Find the **M** independent paths
4. One test case per path

Worked example ✍️ Four atomic conditions give **M = 4 + 1 = 5**, matching **E − N + 2 = 12 − 9 + 2 = 5**. Basis set: P1 qty=0 → 0.0 · P2 price=0 → 0.0 · P3 2×100, not member → 200 · P4 2×100, member, t≤1000 → 200 · P5 20×100, member → 2000×0.9 = 1800.

Independent path A path introducing at least one **new node** or **new condition** no earlier path used. The count equals **M**.

Design in the V-model Business req ↔ **acceptance** · system req ↔ **system test** · architecture ↔ integration · design ↔ unit test.

Nesting & limits statement ⊂ branch ⊂ condition ⊂ path; McCabe's traditional limit is **10**.

Specialised test targets real-time → timing, stress · distributed → latency, partial failure · web → SQL injection, XSS · mobile → devices, screens · embedded → CPU, memory. Full path coverage grows exponentially, so "100 % path coverage" is never a realistic goal.

Night-Before-the-Exam Sheet

Fact sheet

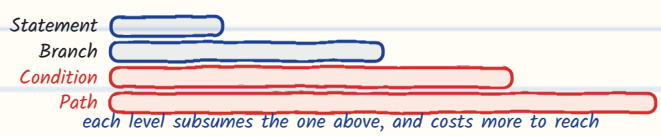
GRASP = General Responsibility Assignment Software Patterns	Carell & Fowler, 1991, IEEE Computer
Guidelines, not a pattern catalogue	Knowing responsibility + doing responsibility
Info Expert = the class with the data	Creator = contains · aggregates · initialises
Low Coupling = least dependent class wins	High Cohesion = one clear reason to change
Polymorphism = do not duplicate in a subclass	Pure Arbiters = arbitrates between two or more
Don't Talk to Strangers = ask only associates	Controller = first non-UI object after the UI
Intermediary / Pure Fabrication = invented class	Protected Variations = interface over change
High Cohesion ≈ SRP · Protected Variations ≈ OCP	Polymorphism ≈ LSP · Intermediary ≈ DIP
Domain model = conceptual, no operations	DCD = adds operations, types, navigability
DCD ↔ code is one-to-one	White box = structure · black box = specification
Testing shows presence of defects, not absence	Statement · Branch · Condition · Path coverage
Multiple-condition coverage needs 2^n combinations	$M = E - N + 2P$; $P = 1$ for a normal module
M = conditions + 1 ; no decision ⇒ $M = 1$	M = independent paths · McCabe's limit 10

→ Top 14 one-liners for MCQs

1. GRASP: **Carell and Fowler, 1991**
2. GRASP guides **responsibility assignment**
3. Information Expert → the class **holding the data**
4. Creator → the class that **contains** the other
5. Low Coupling → the **least dependent** class
6. High Cohesion → **one** clear purpose
7. Controller → the first **non-UI** object
8. Pure Fabrication = **no domain meaning**
9. Protected Variations → an **interface**
10. A DCD shows **operations and navigability**, a domain model does not
11. DCD elements map to code **one-to-one**
12. White box from **structure**; black box from the **spec**
13. **$M = E - N + 2P$** is the cyclomatic number
14. Basis-path needs exactly **M** test cases

Top traps GRASP is not the GoF catalogue · Controller is not the UI class · Creator is not "whoever uses it" · coupling is *between* classes, cohesion *within* one · a domain model has no operations · "white box proves correctness" is wrong · statement coverage is the **weakest** · $M = E - N + 2$ only when $P = 1$ · basis path gives M cases, **not** full path coverage · BVA tests the edges, EP samples the class.

All tricks together "Who has the data?" = Information Expert · "C for Container" = Creator · too many strings = coupling, gravel not a rock = cohesion · a return type means a DCD · white box walks the code, black box reads the spec · M = conditions + 1, and M paths need M cases · EP samples the middle, BVA walks the edge.



the coverage ladder — weakest and cheapest at the top

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. Uses the class that **has the data**:
(a) Creator (b) Controller
(c) Information Expert (d) Low Coupling

2. A **Sale** holds many **SalesLineItems**. Creator?
(a) Sale (b) Register
(c) ProductCatalog (d) Payment

3. Handles an event beyond the UI layer:
(a) Pure Fabrication (b) Controller
(c) Indirection (d) Polymorphism

4. In a DCD but **not** a domain model:
(a) Concept classes (b) Associations
(c) Operations, navigability (d) Multiplicities

5. GRASP was developed by:
(a) Booch & Rumbaugh (b) Carell & Fowler
(c) Beck & Cunningham (d) IEEE

6. Coupling measures dependence:
(a) Within one class (b) Between classes
(c) Between two phases (d) On hardware

7. Cyclomatic number, no decision:
(a) 0 (b) 1
(c) 2 (d) −1

8. $E = 14$, $N = 11$, $P = 1 \Rightarrow M =$:
(a) 3 (b) 4
(c) 5 (d) 23

9. Basis-path cases for $M = 7$:
(a) 6 (b) 7
(c) 8 (d) 14

10. Black-box cases come from:
(a) Source code (b) Flow graph
(c) The specification (d) The CRC card

Answer key → 1-c · 2-a · 3-b · 4-c · 5-b · 6-b · 7-b · 8-c · 9-b · 10-c

Written question 1 Define GRASP, name its authors and year, and explain **Information Expert** and **Creator** with one worked example each. 10 marks

1. **GRASP** = General Responsibility Assignment Software Patterns — guidelines for assigning responsibilities 2
2. **Authors** — James Carell and Martin Fowler, 1991, *Toward Unified Design Methodologies*, IEEE Computer 2
3. **Information Expert** — the class holding the information takes the job; *Invoice.getTotal()*, because the Invoice holds the line items 3
4. **Creator** — the class that contains, aggregates, records, closely uses or can initialise the other creates it; *Sale* creates *SalesLineItem* 3

condition1	condition2	action
Y	Y	accept
Y	N	reject
N	Y	reject
N	N	reject

2 conditions → $2^2 = 4$ rows

cause-effect graphing yields a decision table

Written question 2 A method has two nested if statements and two && conditions. Compute its cyclomatic number and state how many basis-path test cases are needed.

8 marks **Answer:** four atomic conditions ⇒ $M = 4 + 1 = 5$; confirm with $M = E - N + 2P$, then write exactly **five** cases.

→ Before the exam ✓

- ✓ Name GRASP's authors and the year
- ✓ Recite all nine guidelines, one line each
- ✓ Tell a domain model from a DCD in one test
- ✓ Compute M from E, N, P and from conditions
- ✓ Distinguish EP from BVA with numbers

Neighbours: the use-case model and V-model are in **Waterfall & UML**; test levels, debugging and metrics in **Testing Strategies**.